

Name: Ethan Jia

Student ID: ethanjia (2235723)

Section 1: Edge Impulse Results

1. Public Shareable Link:

<https://studio.edgeimpulse.com/public/996586/live>

1. The raw inputs are 3-axis accelerometer time-series signals: accX, accY, and accZ.
2. The NN classifier uses spectral features extracted from the three accelerometer input axes: accX, accY, and accZ. The input time-series window size is 2000 ms, the window increase/stride is 220 ms, and the sampling frequency is 100 Hz. The classifier receives 33 input features from the spectral analysis block.

At least five feature names used to train the classifier include:

accX RMS

accX Peak 1 Freq

accX Peak 1 Height

accX Spectral Power 2.0 - 5.0 Hz

accY RMS

accY Spectral Power 2.0 - 5.0 Hz

accZ Spectral Power 2.0 - 5.0 Hz

3. The NN classifier outputs two classes: nominal and off. Based on the validation results shown in Edge Impulse, the quantized int8 model achieved 100.0% validation accuracy with an F1 score of 1.00 for both classes.

4. The anomaly detector uses K-means and takes selected spectral features as input. The selected features are accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 - 5.0 Hz, and accZ Spectral Power 2.0 - 5.0 Hz. The anomaly detector outputs one feature: the anomaly score.

5.

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 25 ms., Classification: 1 ms., Anomaly: 1 ms.):
  nominal: 0.26172
  off: 0.73828
  anomaly score: 695.302
```

6. The deployment result shows that the Edge Impulse model can classify the fan operating state as either nominal or off while also producing an anomaly score. The validation results show very strong performance, with 100.0% accuracy and F1 scores of 1.00 for both classes. This suggests that the spectral features from the accelerometer data are effective for separating the two fan states in the training and validation data.

However, the classifier output should not always be trusted blindly. It is most reliable when the deployment conditions are similar to the training data, such as the same fan, sensor placement, sampling frequency, mounting method, and operating environment. If the sensor is moved, the fan vibrates differently, there is extra background vibration, or the fan condition is not represented in the training data, the classifier may still output a confident class even when the input is unfamiliar. In those cases, the anomaly score is important because it can indicate that the input does not match the normal training patterns well.

Section 2: Autoencoder Model Results

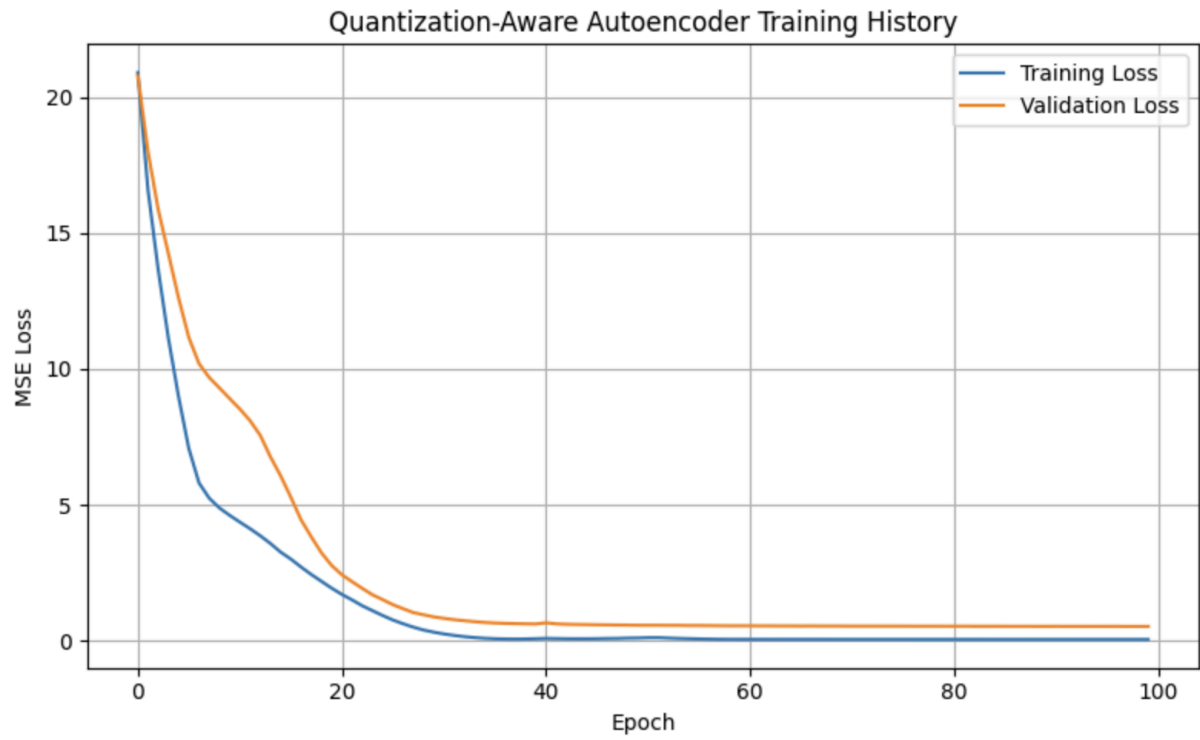


Fig. 1 Screenshot of the training and validation loss curves

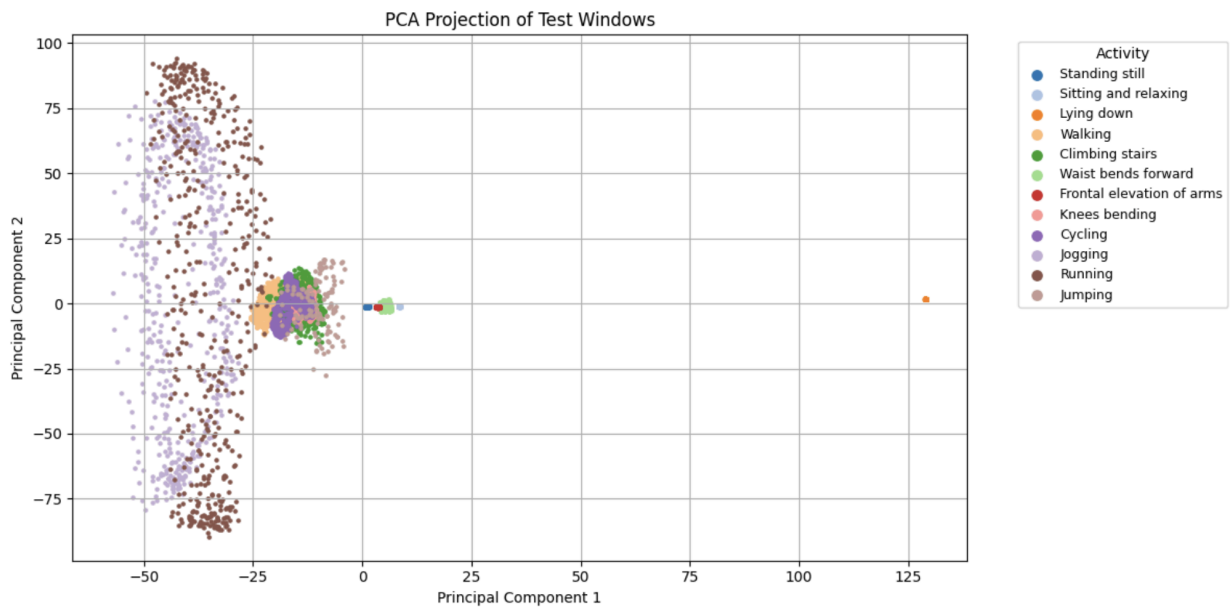


Fig. 2 Screenshot of the PCA visualization

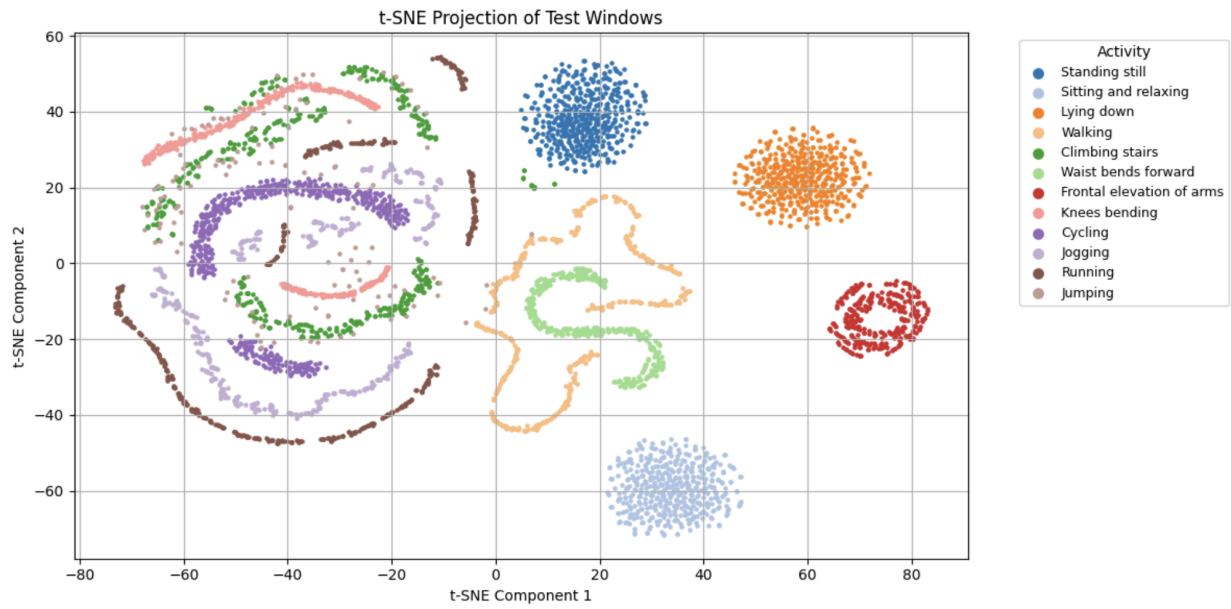


Fig. 3 Screenshot of the t-SNE visualization

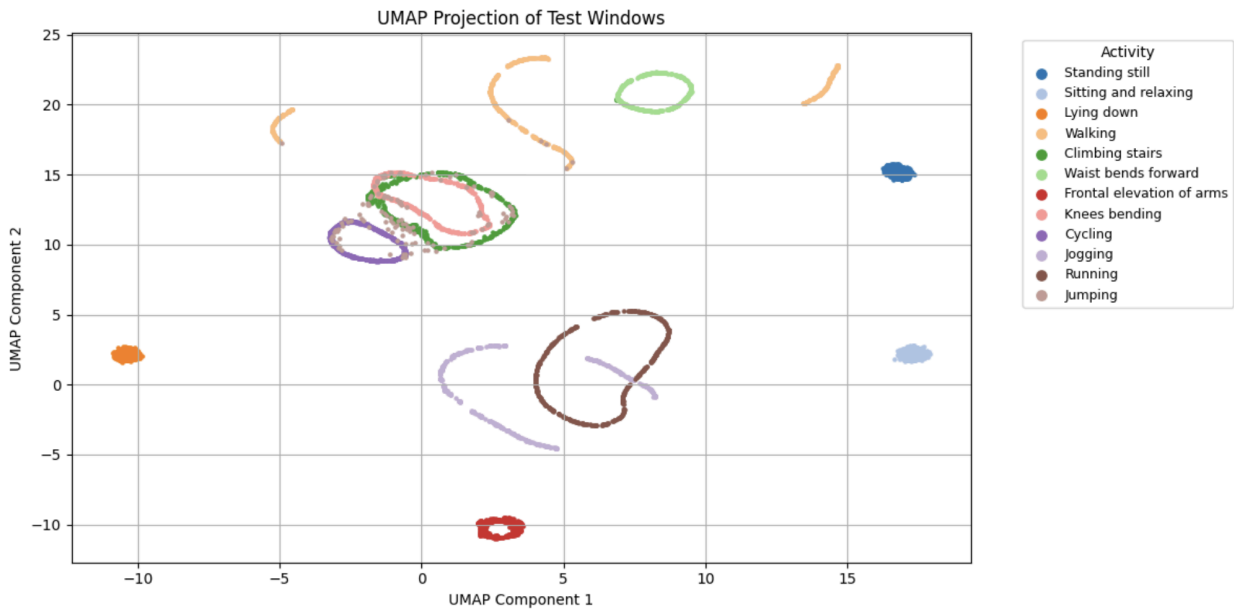


Fig. 4 Screenshot of the UMAP visualization

Selected threshold: 0.512432

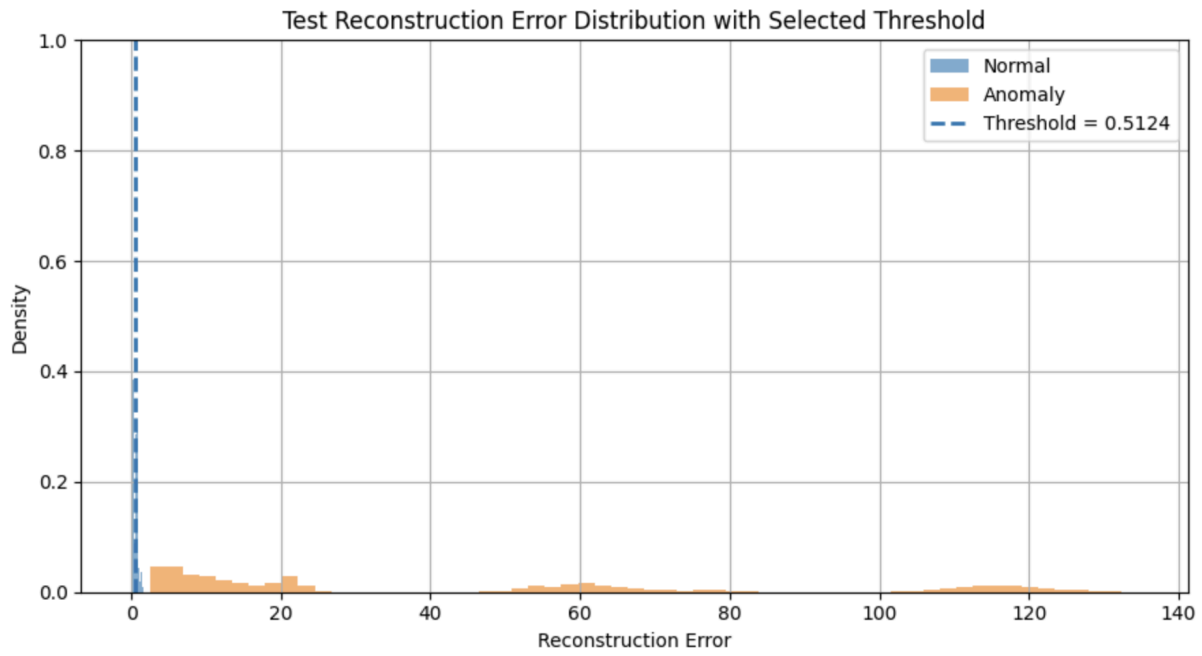
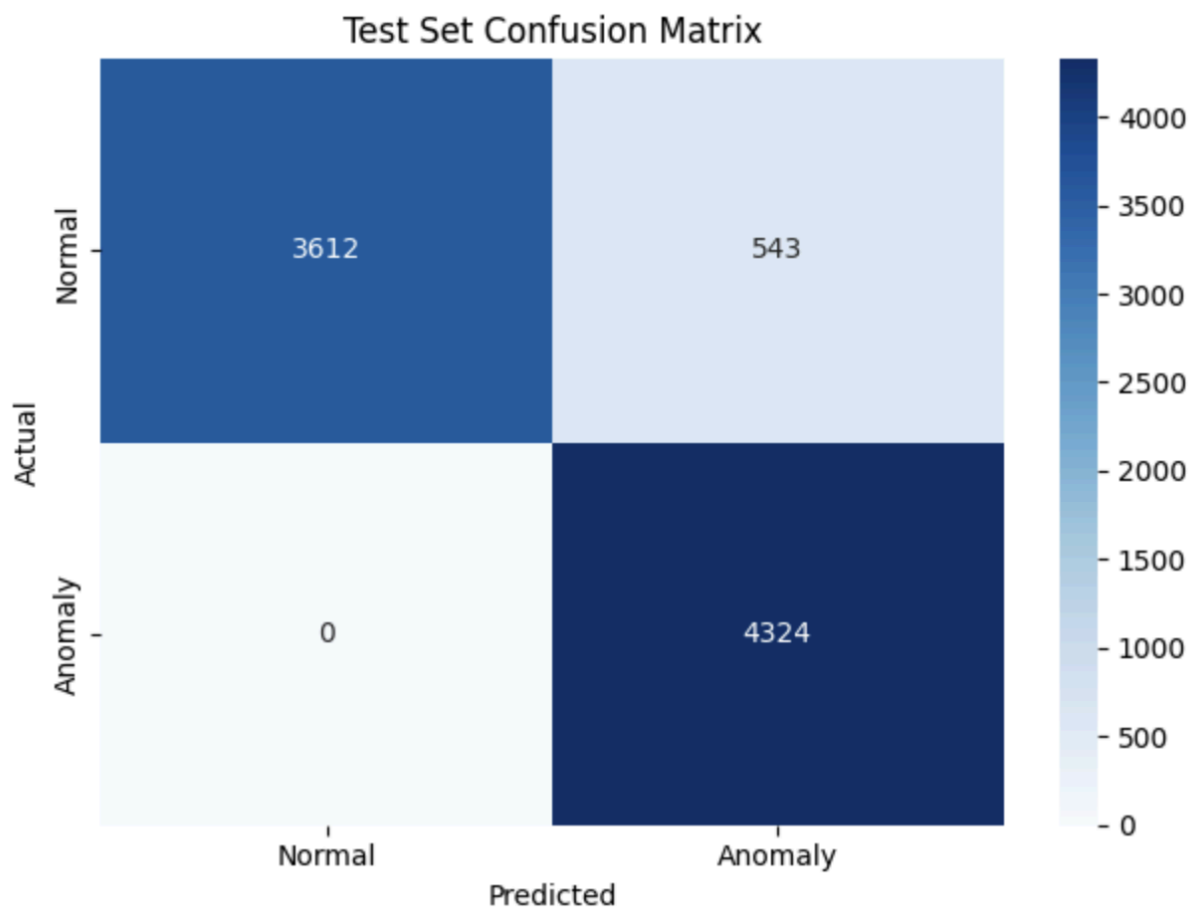


Fig. 5 Screenshot of the Reconstruction error distribution

Selected threshold: 0.512432

Fig. 6 Screenshot of the Selected anomaly threshold



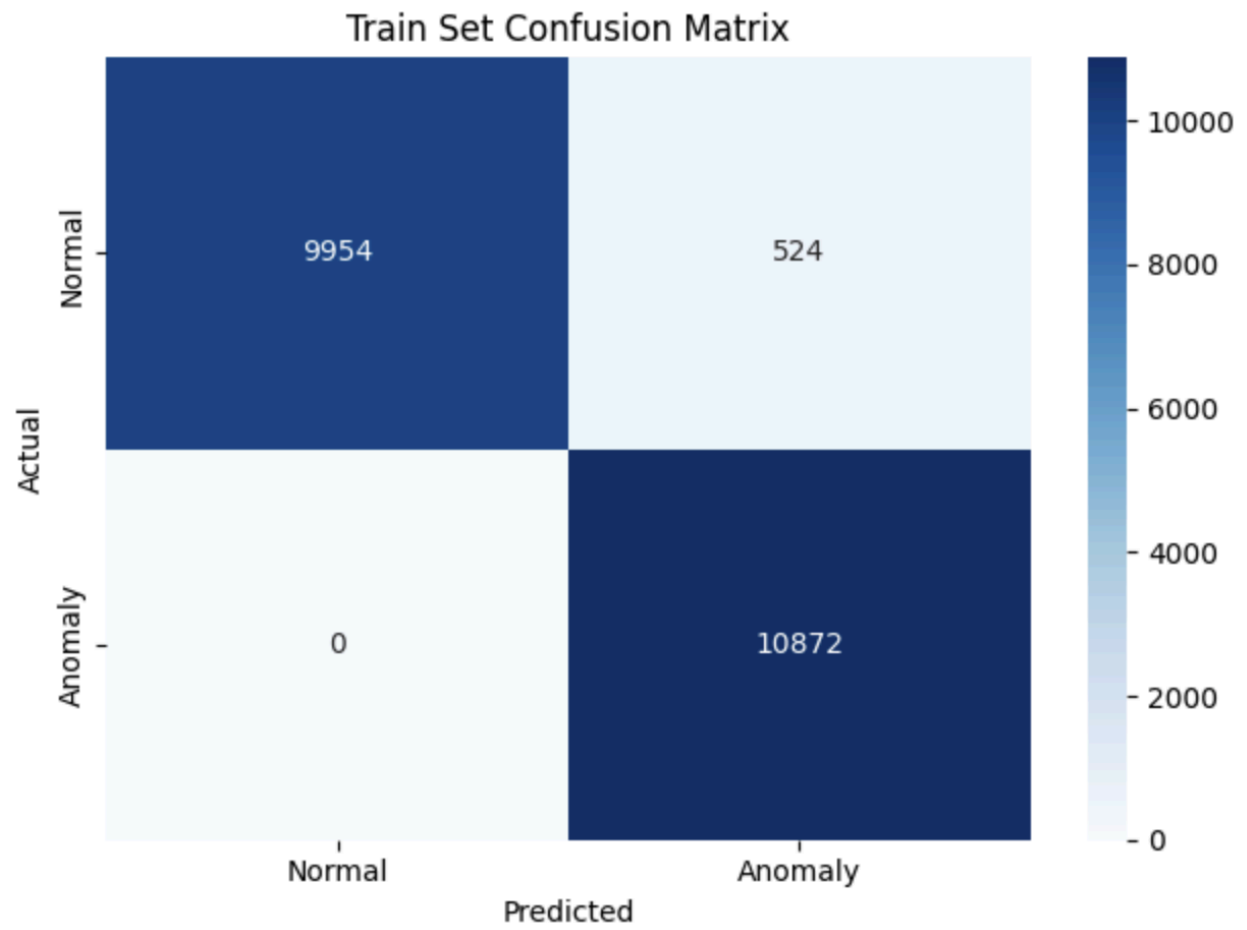


Fig. 7 Screenshot of the Confusion matrix

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.95	0.97	10478
Anomaly (1)	0.95	1.00	0.98	10872
accuracy			0.98	21350
macro avg	0.98	0.97	0.98	21350
weighted avg	0.98	0.98	0.98	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	0.87	0.93	4155
Anomaly (1)	0.89	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.94	0.93	0.94	8479
weighted avg	0.94	0.94	0.94	8479

Fig. 8 Screenshot of the Classification report or evaluation metrics

Reconstruction error: 0.202522 Result: Normal activity Reconstruction error: 0.202511 Result: Normal activity Reconstruction error: 0.202509 Result: Normal activity	Reconstruction error: 3.866709 Result: Anomaly detected Reconstruction error: 3.911656 Result: Anomaly detected Reconstruction error: 3.990546 Result: Anomaly detected
---	--

Fig. 9 Screenshot of the Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

Section 3: Discussion Questions

1. What threshold did you choose for anomaly detection, and why?

I chose a reconstruction error threshold of 0.152432, based on the 95th percentile of the normal training reconstruction errors. This threshold keeps most normal samples below the threshold while allowing samples with unusually high reconstruction error to be classified as anomalies.

2. How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained mainly on normal activity windows, so it learns to reconstruct normal patterns well and produces low reconstruction error for them. Anomalous activities are less familiar to the model, so they are reconstructed less accurately and usually produce higher reconstruction error.

3. What information from the Python notebook must be preserved when deploying the model to Arduino?

The deployed Arduino code must preserve the trained int8 autoencoder model, input window size, feature selection, input dimension, normalization or preprocessing assumptions, reconstruction error calculation, and the selected anomaly threshold.

4. Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The model was trained using a specific input format and preprocessing pipeline. If the Arduino sketch uses different feature columns, scaling, window size, or data ordering, the model input will not match the training data, causing unreliable reconstruction errors and incorrect normal/anomaly predictions.

5. What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The method is limited by the microcontroller's memory, computation speed, and sensor accuracy. It may also produce false positives if normal activity differs from the training data, and it may miss anomalies that are similar to normal patterns. Since the threshold is fixed, performance can degrade when the environment, sensor placement, or user behavior changes.